

PROGRAMACIÓN COMPETITIVA · ICPC

Algorituario ICPC

Referencia rápida de algoritmos para competencia

46

ALGORITMOS

8

CATEGORÍAS

2,378

LÍNEAS DE CÓDIGO

EDICIÓN GENERADA EL 26 DE AGOSTO DE 2026

Índice

Las categorías y los algoritmos están enlazados. Haz clic en cualquier entrada para ir directamente a su página.

Data Structures	4
bint	5
bit_purq	6
bit_rupq	7
dsu	8
monotonic_queue	9
segment_tree	10
sparse_table	11
Dynamic-programming	12
knapsack_01_optimization	13
knuth_optimization	14
lis	15
Flows	16
dinic	17
Min_Cost_Max_flow	22
Geometry	24
circle_line_intersection	25
closest_pair	26
convex_hull	27
formulas	28
Geometry_template_2d	30
Line_intersec	32
point_inside_convex_polygon	33
Point_Inside_Polygon	34
Polygon_Area	35
Graph	36
bellman_ford	37
binary_lifting	38
cycle_detection_directed_graph	39
dijkstra	40

floyd_warshall	41
lca	42
mst	43
tarjan	44
tarjan_bridges_articulation	46
tarjan_scc	47
Math	48
binary_exponentiation	49
combinatorics	50
fft	51
matrix_expo	54
modular_arithmetic	55
precision	56
prime_theory	57
Misc	58
jose	59
random	60
String	61
aho_corasick	62
hashing	63
kmp	64
suffix_array	65
Trie	70
z_funtion	71

SECCIÓN 01

Data Structures



7 algoritmos

bint

DATA STRUCTURES · C++ · 63 LÍNEAS

```
# CÓDIGO · bint

1  const int base {1'000'000'000};
2  class Bint {
3  public:
4      Bint(int num) : d(1, num) {};
5      Bint(std::string& num) {
6          int n{std::ssize(num)};
7          for (int i{n}; i > 0; i -= 9) {
8              d.push_back(std::stoi((i >= 9) ? num.substr(i - 9, 9) : num.substr(0, i)));
9          }
10     }
11     Bint operator+(Bint& o) {
12         Bint bint{}; auto& ans{bint.d}; int carry{};
13         for (int i{}; i < std::max(size(), o.size()); ++i) {
14             auto x{(i < size()) ? d[i] : 0};
15             auto y {(i < o.size()) ? o[i] : 0};
16             auto v {(x + y + carry)};
17             ans.push_back(v % base);
18             carry = (v >= base);
19         }
20         if (carry) ans.push_back(carry);
21         bint.trim();
22         return bint;
23     }
24     Bint operator*(int b) {
25         Bint bint{};
26         auto& ans {bint.d};
27         long long c{};
28         for (int i{}; i < size(); ++i) {
29             long long x {(d[i] * 1ll * b) + c};
30             ans.push_back(x % base);
31             c = (x / base);
32         }
33         while (c) ans.push_back(c % base); c /= base;
34         bint.trim();
35         return bint;
36     }
37     bool operator==(Bint& o) {
38         if (size() != o.size()) return false;
39         for (int i{}; i < size(); ++i) if (d[i] != o[i]) return false;
40         return true;
41     }
42     bool operator<(Bint& o) {
43         if (size() != o.size()) return size() < o.size();
44         for (int i{size() - 1}; i >= 0; --i) if (d[i] != o[i]) return d[i] < o[i];
45         return false;
46     }
47     std::string toString() {
48         if (d.empty()) return "0";
49         std::string s{std::format("{", d.back())};
50         s.reserve(9 * size());
51         for (int i{size() - 2}; i >= 0; --i) {
52             s += std::format("{:09d}", d[i]);
53         }
54         return s;
55     }
56 private:
57     std::vector<int> d{};
58     int operator[](int i) { return d[i]; }
59     int size() { return std::ssize(d); }
60     void trim() {
61         while (size() > 1 && d.back() == 0) d.pop_back();
62     }
63 };
```

bit_purq

DATA STRUCTURES · C++ · 43 LÍNEAS

```
# CÓDIGO · bit_purq

1  class BitPURQ {
2  public:
3      BitPURQ (const std::vector<long long>& a)
4          : n{static_cast<int>(a.size()) + 1}, ft(n) {
5          for (int i{1}; i < n; ++i) {
6              ft[i] += a[i - 1];
7              if (i + lsb(i) < n) ft [i + lsb(i)] += ft[i];
8          }
9      }
10     long long query(int l, int r) {
11         return sum(r) - sum(l - 1);
12     }
13     void increment(int i, long long delta) {
14         while (i < n) {
15             ft[i] += delta;
16             i += lsb(i);
17         }
18     }
19     int find(long long k) {
20         int i{}, b{1};
21         while (b * 2 < n) b *= 2;
22         for (int j{b}; j; j >>= 1) {
23             if (i + j < n && ft[i + j] < k) {
24                 k -= ft[i + j];
25                 i += j;
26             }
27         }
28         return i + 1;
29     }
30 private:
31     int n{};
32     std::vector<long long> ft{};
33     int lsb (int i) { return i&(-i); }
34
35     long long sum(int i) {
36         long long ans{};
37         while (i) {
38             ans += ft[i];
39             i -= lsb(i);
40         }
41         return ans;
42     }
43 };
```

bit_rupq

DATA STRUCTURES · C++ · 32 LÍNEAS

```
# CÓDIGO · bit_rupq

1 class BitRUPQ {
2 public:
3     BitRUPQ (const std::vector<long long>& a)
4         : n{static_cast<int>(a.size()) + 1}, ft(n) {
5             for (int i{}; i < std::ssize(a); ++i) {
6                 increment(i + 1, i + 1, a[i]);
7             }
8         }
9     long long value (int i) {
10         long long ans{};
11         while (i) {
12             ans += ft[i];
13             i -= lsb(i);
14         }
15         return ans;
16     }
17     void increment(int l, int r, long long delta) {
18         sum(l, delta);
19         sum(r + 1, -delta);
20     }
21 private:
22     int n{};
23     std::vector<long long> ft{};
24     int lsb(int i) { return (i&(-i)); }
25
26     void sum(int i, long long delta) {
27         while (i < n) {
28             ft[i] += delta;
29             i += lsb(i);
30         }
31     }
32 };
```

dsu

DATA STRUCTURES · C++ · 28 LÍNEAS

```
# CÓDIGO · dsu

1  class UF {
2  public:
3      UF (int size)
4          : n{size}, c{n}, p(n), sz(n, 1) {
5              std::iota(p.begin(), p.end(), 0);
6          }
7      int find (int i) {
8          while (p[i] != i) i = p[i] = p[p[i]];
9          return i;
10     }
11     bool same(int i, int j) {
12         return find(i) == find(j);
13     }
14     bool unite(int i, int j) {
15         i = find(i); j = find(j);
16         if (i == j) return false;
17         if (sz[i] < sz[j]) std::swap(i, j);
18         p[j] = i;
19         sz[i] += sz[j];
20         --c;
21         return true;
22     }
23     int size(int i) { return sz[find(i)]; }
24     int components() { return c; }
25 private:
26     int n{}, c{};
27     std::vector<int> p{}, sz{};
28 };;
```

monotonic_queue

DATA STRUCTURES · C++ · 23 LÍNEAS

```
# CÓDIGO · monotonic_queue

1 // Monotonic Stack
2 std::vector<int> stk{-1};
3 stk.reserve(n + 1);
4
5 for (int i{}; i < n; ++i) {
6     while (stk.back() != -1 && a[stk.back()] <= a[i]) stk.pop_back();
7     // process
8     stk.push_back(i);
9 }
10
11 // Monotonic deque
12 int m { /* window size */};
13 std::deque<int> dq{};
14
15 for (int i{}; i < n; ++i) {
16     while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back();
17     dq.push_back(i);
18     if (dq.front() <= i - m) dq.pop_front();
19
20     if (i >= m - 1) {
21         int max_value {a[dq.front()]};
22     }
23 }
```

segment_tree

DATA STRUCTURES · C++ · 59 LÍNEAS

```
# CÓDIGO · segment_tree

1 class SegmentTree {
2     using T = long long;
3 public:
4     SegmentTree (const std::vector<T>& a)
5         : n{static_cast<int>(a.size())}, tree(n * 4, kdefault), lazy(n * 4, klazy) {
6         build(0, 0, n - 1, a);
7     }
8     void increment (int l, int r, T delta);
9     T query (int l, int r);
10 private:
11     int n{};
12     const T kdefault{}, klazy{};
13     std::vector<T> tree{}, lazy{};
14
15     T agg (T a, T b) { return std::max(a, b); }
16     int left (int node) { return (node << 1) + 1; }
17     int right (int node) { return (node << 1) + 2; }
18     int mid (int a, int b) { return (a + b) / 2; }
19
20     void build(int node, int l, int r, const std::vector<T>& a) {
21         if (l == r) {
22             tree[node] = a[l];
23             return;
24         }
25         build(left(node), l, mid(l, r), a);
26         build(right(node), mid(l, r) + 1, r, a);
27         tree[node] = agg(tree[left(node)], tree[right(node)]);
28     }
29     void increment(int node, int l, int r, int leftq, int rightq, T delta) {
30         propagate(node, l, r);
31         if (r < leftq || rightq < l) return;
32         if (leftq <= l && r <= rightq) {
33             lazy[node] += delta;
34             propagate(node, l, r);
35             return;
36         }
37         increment(left(node), l, mid(l, r), leftq, rightq, delta);
38         increment(right(node), mid(l, r) + 1, r, leftq, rightq, delta);
39         tree[node] = agg(tree[left(node)], tree[right(node)]);
40     }
41     T query(int node, int l, int r, int leftq, int rightq) {
42         if (r < leftq || rightq < l) return kdefault;
43         propagate(node, l, r);
44         if (leftq <= l && r <= rightq) return tree[node];
45         return agg (
46             query(left(node), l, mid(l, r), leftq, rightq),
47             query(right(node), mid(l, r) + 1, r, leftq, rightq)
48         );
49     }
50     void propagate (int node, int l, int r) {
51         if (lazy[node] == klazy) return;
52         tree[node] += lazy[node];
53         if (l != r) {
54             lazy[left(node)] += lazy[node];
55             lazy[right(node)] += lazy[node];
56         }
57         lazy[node] = klazy;
58     }
59 };
```

sparse_table

DATA STRUCTURES · C++ · 26 LÍNEAS

CÓDIGO · sparse_table

```
1  class SparseTable {
2  public:
3      SparseTable(const std::vector<int>& b)
4          : n {static_cast<int>(b.size())}, lg(n + 1) {
5          for (int i{2}; i <= n; ++i) lg[i] = lg[i / 2] + 1;
6          m = lg[n] + 1;
7
8          a.resize(m * n);
9          for (int i{}; i < n; ++i) v(i, 0) = b[i];
10
11         for (int j{1}; j < m; ++j) {
12             for (int i{}; i + (1 << j) - 1 < n; ++i) {
13                 v(i, j) = std::min(v(i, j - 1), v(i + (1 << (j - 1)), j - 1));
14             }
15         }
16     }
17     int query(int l, int r) {
18         --l; --r;
19         int k {lg[r - l + 1]};
20         return std::min(v(l, k), v(r - (1 << k) + 1, k));
21     }
22 private:
23     int n{}, m{};
24     std::vector<int> a{}, lg{};
25     int& v(int i, int j) { return a[(j * n) + i]; }
26 };
```

SECCIÓN 02

Dynamic-programming



3 algoritmos

knapsack_01_optimization

DYNAMIC-PROGRAMMING · C++ · 24 LÍNEAS

```
# CÓDIGO · knapsack_01_optimization

1  /* 0/1 Knapsack optimization where sum of all items is ~ N
2     There will be at most sqrt(N) different items.
3     Array 'cnt' represents the count of a specific item.
4  */
5
6  const long long maxn = 1e5+50;
7  const long long maxnsq = 400;
8
9  long long n,m,cnt[maxn],dp[maxnsq][maxn];
10 vector<long long> c;
11 void calculateDp(){ //DP in O(N*sqrt(N))
12     dp[0][0]=0;
13     for(int i = 1; i < n+5; i++) dp[0][i] = -1;
14     for(int i = 1; i < (int)c.size(); i++){ // c is the array of unique items.
15         for(int j = 1; j < n+1; j++){
16             if(dp[i-1][j] >= 0)
17                 dp[i][j] = 0;
18             else if(j-c[i] >= 0 && dp[i][j - c[i]] >= 0 and dp[i][j - c[i]] < cnt[c[i]])
19                 dp[i][j] = dp[i][j - c[i]] + 1;
20             else
21                 dp[i][j] = -1;
22         }
23     }
24 }
```

knuth_optimization

DYNAMIC-PROGRAMMING · C++ · 45 LÍNEAS

```
# CÓDIGO · knuth_optimization

1  /*
2  Optimization:
3  Special case of DP on ranges, that optimizes the time
4  complexity of solutions by a linear factor, from  $O(n^3)$ 
5  to  $O(n^2)$ .
6
7  Recurrence:
8   $dp(i,j) = \text{fore}(k,i,j) \min[dp(i,k) + dp(k+1,j) + C(i,j)]$ 
9
10 We can apply this optimization when the cost function C
11 satisfies the following conditions for  $a \leq b \leq c \leq d$ :
12 1.  $C(b,c) \leq C(a,d)$ 
13 2.  $C(a,c) + C(b,d) \leq C(a,d) + C(b,c)$  (quad. ineq.)
14 */
15
16 int solve() {
17     int N;
18     ... // read N and input
19     int dp[N][N], opt[N][N];
20
21     auto C = [&](int i, int j) {
22         ... // Implement cost function C.
23     };
24
25     for (int i = 0; i < N; i++) {
26         opt[i][i] = i;
27         ... // Initialize dp[i][i] according to the problem
28     }
29
30     for (int i = N-2; i >= 0; i--) {
31         for (int j = i+1; j < N; j++) {
32             int mn = INT_MAX;
33             int cost = C(i, j);
34             for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]); k++) {
35                 if (mn >= dp[i][k] + dp[k+1][j] + cost) {
36                     opt[i][j] = k;
37                     mn = dp[i][k] + dp[k+1][j] + cost;
38                 }
39             }
40             dp[i][j] = mn;
41         }
42     }
43
44     return dp[0][N-1];
45 }
```

lis

DYNAMIC-PROGRAMMING · C++ · 16 LÍNEAS

```
# CÓDIGO · lis
1 // Longest increasing subsequence O(nlogn)
2 const long long INF = 1e18;
3 int lis(const vector<long long> &a) {
4     int n = (int)a.size();
5     vector<long long> d(n+1, INF);
6     d[0] = -INF;
7
8     for (int i = 0; i < n; i++) {
9         int j = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
10        if (d[j-1] < a[i] && a[i] < d[j]) d[j] = a[i];
11    }
12
13    int ans = 0;
14    for(int i = 0; i < n+1; i++) if (d[i]<INF) ans = i;
15    return ans;
16 }
```

SECCIÓN 03

Flows



2 algoritmos

dinic

FLAWS · C++ · 341 LÍNEAS

```
# CÓDIGO · dinic

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 using ll = long long;
6
7 struct FlowEdge {
8     int from;
9     int to;
10    ll cap;
11    ll flow;
12
13    FlowEdge(int from, int to, ll cap)
14        : from(from), to(to), cap(cap), flow(0) {}
15 };
16
17 struct Dinic {
18     const ll INF = (ll)4e18;
19
20     int n;
21     int source;
22     int sink;
23
24     vector<FlowEdge> edges;
25     vector<vector<int>> adj;
26
27     vector<int> level;
28     vector<int> ptr;
29
30     Dinic(int n, int source, int sink)
31         : n(n), source(source), sink(sink) {
32
33         adj.resize(n);
34         level.resize(n);
35         ptr.resize(n);
36     }
37
38     // Directed edge: from -> to with capacity cap
39     void add_edge(int from, int to, ll cap) {
40         int id = (int)edges.size();
41
42         // Original edge
43         edges.emplace_back(from, to, cap);
44
45         // Residual edge
46         edges.emplace_back(to, from, 0);
47
48         adj[from].push_back(id);
49         adj[to].push_back(id + 1);
50     }
51
52     // Builds the level graph
53     bool bfs() {
54         fill(level.begin(), level.end(), -1);
55
56         queue<int> q;
57
58         level[source] = 0;
59         q.push(source);
60
61         while (!q.empty()) {
62             int v = q.front();
63             q.pop();
64
65             for (int id : adj[v]) {
66                 FlowEdge& e = edges[id];
```

```

# CÓDIGO · dinic

67
68         // No residual capacity
69         if (e.cap - e.flow <= 0) {
70             continue;
71         }
72
73         // Already visited
74         if (level[e.to] != -1) {
75             continue;
76         }
77
78         level[e.to] = level[v] + 1;
79         q.push(e.to);
80     }
81 }
82
83     return level[sink] != -1;
84 }
85
86 // Sends flow through the level graph
87 ll dfs(int v, ll pushed) {
88     if (pushed == 0) {
89         return 0;
90     }
91
92     if (v == sink) {
93         return pushed;
94     }
95
96     for (int& cid = ptr[v];
97          cid < (int)adj[v].size();
98          ++cid) {
99
100         int id = adj[v][cid];
101         FlowEdge& e = edges[id];
102
103         if (level[e.to] != level[v] + 1) {
104             continue;
105         }
106
107         if (e.cap - e.flow <= 0) {
108             continue;
109         }
110
111         ll tr = dfs(
112             e.to,
113             min(pushed, e.cap - e.flow)
114         );
115
116         if (tr == 0) {
117             continue;
118         }
119
120         e.flow += tr;
121
122         // Reverse edge
123         edges[id ^ 1].flow -= tr;
124
125         return tr;
126     }
127
128     return 0;
129 }
130
131 // Returns maximum flow
132 ll max_flow() {
133     ll totalFlow = 0;
134
135     while (bfs()) {
136         fill(ptr.begin(), ptr.end(), 0);
137
138         while (true) {

```

```

# CÓDIGO · dinic

139         ll pushed = dfs(source, INF);
140
141         if (pushed == 0) {
142             break;
143         }
144
145         totalFlow += pushed;
146     }
147 }
148
149 return totalFlow;
150 }
151 };
152
153 int main() {
154     ios_base::sync_with_stdio(false);
155     cin.tie(nullptr);
156
157     /*
158     BASIC USAGE
159     -----
160
161     Suppose we have nodes:
162
163         0, 1, 2, 3
164
165     plus:
166
167         source = 4
168         sink   = 5
169
170     Therefore:
171
172         totalNodes = 6
173     */
174
175     int totalNodes = 6;
176     int source = 4;
177     int sink = 5;
178
179     Dinic dinic(totalNodes, source, sink);
180
181     /*
182     Add directed edges:
183
184     add_edge(from, to, capacity)
185     */
186
187     dinic.add_edge(source, 0, 3);
188     dinic.add_edge(source, 1, 2);
189
190     dinic.add_edge(0, 2, 2);
191     dinic.add_edge(0, 3, 1);
192     dinic.add_edge(1, 2, 1);
193     dinic.add_edge(1, 3, 2);
194
195     dinic.add_edge(2, sink, 2);
196     dinic.add_edge(3, sink, 3);
197
198     /*
199     Run Dinic.
200     */
201
202     ll flow = dinic.max_flow();
203
204     cout << "Maximum flow: " << flow << '\n';
205
206
207     /*
208     -----
209     HOW TO INSPECT THE FLOW OF EVERY EDGE
210     -----

```

```

# CÓDIGO · dinic

211
212     Every add_edge() adds TWO edges:
213
214         edges[id] = original
215         edges[id ^ 1] = residual
216
217     Therefore, original edges are:
218
219         0, 2, 4, 6, ...
220
221     You can recover which edges were used by
222     checking:
223
224         e.flow > 0
225 */
226
227 for (int id = 0; id < (int)dinic.edges.size(); id += 2) {
228     FlowEdge& e = dinic.edges[id];
229
230     cout
231         << e.from
232         << " -> "
233         << e.to
234         << "    flow = "
235         << e.flow
236         << "/"
237         << e.cap
238         << '\n';
239 }
240
241
242 /*
243 -----
244 BIPARTITE MATCHING
245 -----
246
247 If you have:
248
249     Left side:
250         0 ... L - 1
251
252     Right side:
253         L ... L + R - 1
254
255     source = L + R
256     sink   = L + R + 1
257
258 then:
259
260     Dinic dinic(L + R + 2, source, sink);
261
262     source -> LEFT:
263         capacity 1
264
265     LEFT -> RIGHT:
266         capacity 1
267
268     RIGHT -> sink:
269         capacity 1
270
271
272 Example:
273
274     for (int i = 0; i < L; ++i) {
275         dinic.add_edge(source, i, 1);
276     }
277
278     for (int j = 0; j < R; ++j) {
279         dinic.add_edge(L + j, sink, 1);
280     }
281
282     if (left_i can match right_j) {

```

```

# CÓDIGO · dinic

283         dinic.add_edge(i, L + j, 1);
284     }
285
286     ll matching = dinic.max_flow();
287
288
289     -----
290     RECOVER MATCHES
291     -----
292
293     for (int id = 0;
294         id < (int)dinic.edges.size();
295         id += 2) {
296
297         FlowEdge& e = dinic.edges[id];
298
299         bool fromLeft =
300             e.from >= 0 &&
301             e.from < L;
302
303         bool toRight =
304             e.to >= L &&
305             e.to < L + R;
306
307         if (fromLeft && toRight && e.flow == 1) {
308
309             int left = e.from;
310             int right = e.to - L;
311
312             cout << left << " " << right << '\n';
313         }
314     }
315
316
317     -----
318     B-MATCHING
319     -----
320
321     If every node can be matched K times,
322     just change the capacities:
323
324         source -> left      capacity K
325         right  -> sink      capacity K
326
327     while:
328
329         left -> right      capacity 1
330
331     Example for degree <= 2:
332
333         dinic.add_edge(source, left, 2);
334         dinic.add_edge(right, sink, 2);
335
336     The reconstruction is exactly the same:
337     left -> right edges with flow == 1.
338 */
339
340     return 0;
341 }

```

Min_Cost_Max_flow

FLAWS · C++ · 84 LÍNEAS

```
# CÓDIGO · Min_Cost_Max_flow

1  const long long inf = 1e18+7;
2
3  struct FlowGraph {
4
5      struct Edge {
6          long long to, flow, cap, cost;
7          Edge *res;
8
9          Edge () : to(0), flow(0), cap(0), cost(0), res(0) {}
10         Edge (long long to, long long flow, long long cap, long long cost) : to(to), flow(flow), cap(cap),
11             cost(cost), res(0) {}
12
13         void addFlow (long long f) {
14             flow += f;
15             res->flow -= f;
16         }
17     };
18
19     long long n;
20     vector<vector<Edge*>> adj;
21     vector<long long> dis, pos;
22
23     FlowGraph (int n) : n(n), adj(n), dis(n), pos(n) {}
24
25     void add (int u, int v, long long cap, long long cost) {
26         Edge *x = new Edge(v, 0, cap, cost);
27         Edge *y = new Edge(u, cap, cap, -cost);
28         x->res = y;
29         y->res = x;
30         adj[u].push_back(x);
31         adj[v].push_back(y);
32     }
33
34     pair<long long, long long> mcmf(int s, int t, long long tope) {
35         vector<bool> inq(n);
36         vector<long long> dis(n), cap(n);
37         vector<Edge*> par(n);
38         long long maxFlow = 0, minCost = 0;
39
40         while (maxFlow < tope) { // compute MCF: maxflow < tope, compute MCMF maxflow < inf
41             fill(dis.begin(), dis.end(), inf);
42             fill(par.begin(), par.end(), nullptr);
43             fill(cap.begin(), cap.end(), 0);
44             dis[s] = 0;
45             cap[s] = inf;
46             queue<int> q;
47             q.push(s);
48
49             while ((int)q.size()) {
50                 int u = q.front();
51                 q.pop();
52                 inq[u] = 0;
53
54                 for (Edge *v : adj[u]) {
55                     if (v->cap > v->flow && dis[v->to] > dis[u] + v->cost) {
56                         dis[v->to] = dis[u] + v->cost;
57                         par[v->to] = v;
58                         cap[v->to] = min(cap[u], v->cap - v->flow);
59
60                         if (!inq[v->to]) {
61                             q.push(v->to);
62                             inq[v->to] = 1;
63                         }
64                     }
65                 }
66             }
67         }
68     }
69 }
```

CÓDIGO · Min_Cost_Max_flow

```
67
68     if (!par[t]) break;
69
70     maxFlow += cap[t];
71     minCost += cap[t] * dis[t];
72     for (int u = t; u != s; u = par[u]->res->to)
73         par[u]->addFlow(cap[t]);
74 }
75
76 return {maxFlow, minCost};
77 }
78 };
79
80 void doit(){
81     // define src and sink.
82     // edges src to node, and node to sink have cost 0.
83     // to compute flow matches (e.g assignment problems), run dfs over the flow graph, keep the path and
84     // subtract one unit of flow every time.
85 }
```

SECCIÓN 04

Geometry



9 algoritmos

circle_line_intersection

GEOMETRY · C++ · 19 LÍNEAS

```
# CÓDIGO · circle_line_intersection

1 vector <Point> circleLineIntersection(double a, double b, double c, double r){
2     //Dados los coeficientes de la ecuacion de la recta y el radio del circulo con centro en el origen
3     vector <Point> pts;
4     double x0 = -a*c/(a*a+b*b), y0 = -b*c/(a*a+b*b);
5     if (c*c > r*r*(a*a+b*b)+EPS){} // 0 points.
6     else if (abs (c*c - r*r*(a*a+b*b)) < EPS) pts.pb({x0,y0}); // 1 point.
7     else{ // 2 points.
8         double d = r*r - c*c/(a*a+b*b);
9         double mult = sqrt (d / (a*a+b*b));
10        double ax, ay, bx, by;
11        ax = x0 + b * mult;
12        bx = x0 - b * mult;
13        ay = y0 - a * mult;
14        by = y0 + a * mult;
15        pts.pb({ax,ay});
16        pts.pb({bx,by});
17    }
18    return pts;
19 }
```

closest_pair

GEOMETRY · C++ · 31 LÍNEAS

CÓDIGO · closest_pair

```
1  #define pii pair<ll,ll>
2
3  ll closestPair(vector<pii> pts){
4      //Calcula el par de puntos en 2D mas cercanos entre si, retorna su distancia euclidiana.
5      int n = sz(pts);
6      sort(ALL(pts));
7      set<pii> s;
8
9      ll ans = INF;
10     int pos = 0;
11     for(i,0,n){
12         ll d = ceil(sqrt(ans));
13         while (pts[i].first - pts[pos].first >= d) {
14             s.erase({pts[pos].second, pts[pos].first});
15             pos++;
16         }
17
18         auto it1 = s.lower_bound({pts[i].second - d, pts[i].first});
19         auto it2 = s.upper_bound({pts[i].second + d, pts[i].first});
20
21         for (auto it = it1; it != it2; it++) {
22             ll dx = pts[i].first - it->second;
23             ll dy = pts[i].second - it->first;
24             if (ans > 1LL * dx * dx + 1LL * dy * dy){
25                 ans = 1LL * dx * dx + 1LL * dy * dy;
26             }
27         }
28         s.insert({pts[i].second, pts[i].first});
29     }
30     return ans;
31 }
```

convex_hull

GEOMETRY · C++ · 23 LÍNEAS

CÓDIGO · convex_hull

```
1 vector <Point> calculateHull(vector <Point> &p, int n){ //Calculo del Convex Hull
2     if (n <= 2) return p;
3     vector<Point> hull;
4     int tam = 0;
5     sort(p.begin(),p.end());
6     for(t,0,2){
7         for(i,0,n){
8             while((int)hull.size()-tam >= 2){
9                 Point p1 = hull[(int)hull.size()-2];
10                Point p2 = hull[(int)hull.size()-1];
11                //Producto cruz: P1 es el ancla
12                //agregar (<=) si tambien se quieren incluir los puntos colineales, sino solo (<)
13                if(p1.cross(p2, p[i]) <= 0) break;
14                hull.pop_back();
15            }
16            hull.push_back(p[i]);
17        }
18        hull.pop_back();
19        tam = (int)hull.size();
20        reverse(p.begin(),p.end());
21    }
22    return hull;
23 }
```

formulas

GEOMETRY · C++ · 79 LÍNEAS

#	CÓDIGO · formulas
1	// Volume of a sphere.
2	ld volumeSphere(ld rad){
3	return (4.0/3.0)*PI*rad*rad*rad;
4	}
5	
6	// Volume of a sphere cap.
7	ld volumeCap(ld h, ld rad){
8	return PI*h*h*(rad-(h/3.0));
9	}
10	
11	// Area of a triangle given vertices A, B, and C
12	ld areaTriangle(Point A, Point B, Point C) {
13	return fabs((A.x * (B.y - C.y) + B.x * (C.y - A.y) + C.x * (A.y - B.y)) / 2.0);
14	}
15	
16	// Area of a circle
17	ld areaCircle(ld radius) {
18	return PI * radius * radius;
19	}
20	
21	// Area of a trapezoid given bases and height
22	ld areaTrapezoid(ld base1, ld base2, ld height) {
23	return 0.5 * (base1 + base2) * height;
24	}
25	
26	// Volume of a cone
27	ld volumeCone(ld radius, ld height) {
28	return (PI * radius * radius * height) / 3.0;
29	}
30	
31	// Volume of a cylinder
32	ld volumeCylinder(ld radius, ld height) {
33	return PI * radius * radius * height;
34	}
35	
36	// Volume of a rectangular prism
37	ld volumeRectPrism(ld length, ld width, ld height) {
38	return length * width * height;
39	}
40	
41	// Volume of a pyramid with a rectangular base
42	ld volumePyramid(ld length, ld width, ld height) {
43	return (length * width * height) / 3.0;
44	}
45	
46	// Area of a parallelogram given two vectors (base and height)
47	ld areaParallelogram(Point base, Point heightVec) {
48	return fabs(base * heightVec);
49	}
50	
51	// Perimeter of a polygon given vertices (assuming vertices are in order)
52	ld perimeterPolygon(vector<Point> &vertices) {
53	ld perimeter = 0.0;
54	for(i, 0, sz(vertices)) {
55	perimeter += vertices[i].dist(vertices[(i + 1) % sz(vertices)]);
56	}
57	return perimeter;
58	}
59	
60	// Volume of a prism with base area and height
61	ld volumePrism(ld baseArea, ld height) {
62	return baseArea * height;
63	}
64	
65	// Surface area of a sphere
66	ld surfaceAreaSphere(ld radius) {

```
# CÓDIGO · formulas

67     return 4 * PI * radius * radius;
68 }
69
70 // Surface area of the cap of a sphere
71 // Note: the radius is the radius of the whole sphere.
72 ld capSurfaceAreaSphere(ld height, ld radius){
73     return 2 * PI * height * radius;
74 }
75
76 // Surface area of a cylinder
77 ld surfaceAreaCylinder(ld radius, ld height) {
78     return 2 * PI * radius * (radius + height);
79 }
```

Geometry_template_2d

GEOMETRY · C++ · 72 LÍNEAS

```
# CÓDIGO · Geometry_template_2d

1 #include <bits/stdc++.h>
2 #define ll long long
3 #define ld long double
4 #define PI acos(-1.0)
5
6 using namespace std;
7
8 struct Point {
9     ld x, y;
10     Point operator +(const Point& b) const { return Point{x+b.x, y+b.y}; }
11     Point operator -(const Point& b) const { return Point{x-b.x, y-b.y}; }
12     ll operator *(const Point& b) const { return (ll) x * b.y - (ll) y * b.x; }
13     Point operator *(const ld k) const { return Point{x*k, y*k}; }
14     bool operator <(const Point& b) const { return x == b.x ? y < b.y : x < b.x; }
15     void operator +=(const Point& b) { x += b.x; y += b.y; }
16     void operator -=(const Point& b) { x -= b.x; y -= b.y; }
17     void operator *=(const int k) { x *= k; y *= k; }
18     bool operator ==(const Point& b) {
19         if (b.x == (*this).x && b.y == (*this).y) return true;
20         return false;
21     }
22     ld magnitude() const { return sqrt((x*x) + (y*y)); }
23     ld dot (const Point &b){ return (x * b.x) + (y * b.y); }
24     // If dot product > 0, angle between vectors is acute, = 0 vectors are perpendicular, and if < 0, angle
    is obtuse
25     ld dist (const Point &b){return (*this - b).magnitude();}
26     ll cross(const Point& b, const Point& c) const {
27         ll cruz = (b - *this) * (c - *this);
28         if (cruz < 0) return -1; //cw (right)
29         if (cruz > 0) return +1; //ccw (left)
30         return 0; //Collinear.
31     }
32     ld rawCross(const Point &a, const Point &b) const {
33         return (a - *this) * (b - *this);
34     }
35     bool onSegment(Point p, Point r){ //point in a rectangular box
36         if ((*this).x <= max(p.x, r.x) && (*this).x >= min(p.x, r.x) && (*this).y <= max(p.y, r.y) && (*
            this).y >= min(p.y, r.y)) return true;
37         return false;
38     }
39     ld angleBetweenVectors(const Point &b){ //this: (b-a), Point b: (c-a).
40         ld ang = acos((*this).dot(b)/((*this).magnitude() * b.magnitude()));
41         ang = (ang * 180.0) / PI;
42         return ang; //return angle in degrees.
43     }
44     int half() const { // for angular sorting, to know in which half is this
45         if ((*this).y > 0) return 0;
46         if ((*this).y < 0) return 1;
47         return (*this).x >= 0 ? 0 : 1;
48     }
49 };
50 struct LineToPoint{
51     Point p1,p2;
52     ld dist(Point refPoint){
53         return abs((refPoint - p1) * (refPoint - p2)) / p1.dist(p2);
54     }
55 };
56 ld degreesToRadians(ld degrees) { return degrees * PI / 180.0; }
57 ld radiansToDegrees(ld radians){ return radians * (180.0 / PI); }
58 Point rotate45(Point p){ // Rotates a point 45 degrees
59     return {p.x + p.y, p.y - p.x};
60 }
61 Point undorotate45(Point p){ // Undo 45 degrees rotation of a point
62     return {(p.x-p.y)/2, (p.y+p.x)/2};
63 }
64 bool angleCmp(const Point& a, const Point& b) { // for angular sorting
65     int ha = a.half();
```

```
# CÓDIGO · Geometry_template_2d

66     int hb = b.half();
67     if (ha != hb) return ha < hb;           // upper half first
68     ll cr = a*b;
69     if (cr != 0) return cr > 0;             // CCW order
70     // Collinear: tie-break by distance from origin (shorter first, optional)
71     return a.magnitude() < b.magnitude();
72 }
```

Line_intersec

GEOMETRY · C++ · 14 LÍNEAS

```
# CÓDIGO · Line_intersec

1  bool doIntersect(Point p1, Point q1, Point p2, Point q2){ //Checa si 2 lineas se intersectan o no.
2      int o1 = p1.cross(q1, p2);
3      int o2 = p1.cross(q1, q2);
4      int o3 = p2.cross(q2, p1);
5      int o4 = p2.cross(q2, q1);
6
7      if (o1 != o2 && o3 != o4) return true;
8      if (o1 == 0 && p2.onSegment(p1, q1)) return true;
9      if (o2 == 0 && q2.onSegment(p1, q1)) return true;
10     if (o3 == 0 && p1.onSegment(p2, q2)) return true;
11     if (o4 == 0 && q1.onSegment(p2, q2)) return true;
12
13     return false;
14 }
```

point_inside_convex_polygon

GEOMETRY · C++ · 61 LÍNEAS

```
# CÓDIGO · point_inside_convex_polygon

1 //Este código asume orden antihorario
2
3 int sgn(ll val) { return val > 0 ? 1 : (val == 0 ? 0 : -1); }
4
5 bool pointInTriangle(Point a, Point b, Point c, Point point){
6     ll s1 = abs(a.rawCross(b, c));
7     ll s2 = abs(point.rawCross(a, b)) + abs(point.rawCross(b, c)) + abs(point.rawCross(c, a));
8     return s1 == s2;
9 }
10
11 //Precalculation for queries to know if a point lies inside of a convex polygon.
12 void prepareConvexPolygon(int &n, vector<Point> &points, vector<Point> &seq, Point &translation){ //seq and
    translation are empty here.
13     n = points.size();
14     int pos = 0;
15     for (int i = 1; i < n; i++) {
16         if (points[i] < points[pos])
17             pos = i;
18     }
19     rotate(points.begin(), points.begin() + pos, points.end());
20
21     n--;
22     seq.resize(n);
23     for (int i = 0; i < n; i++)
24         seq[i] = points[i + 1] - points[0];
25     translation = points[0];
26 }
27
28 //Know if a point lies inside of a convex polygon in O(logN)
29 bool pointInConvexPolygon(Point point, int &n, vector<Point> &seq, Point &translation) {
30     point = point - translation;
31     if (seq[0]*point != 0 && sgn(seq[0]*point) != sgn(seq[0]*seq[n-1])) return false;
32     if (seq[n - 1]*point != 0 && sgn(seq[n - 1]*point) != sgn(seq[n - 1]*seq[0])) return false;
33     if (seq[0]*point == 0) return seq[0].dot(seq[0]) >= point.dot(point);
34     /*
35     Si no acepta puntos en el borde reemplaza linea anterior con
36     if (seq[0] * point == 0) return false;
37     if (seq[n - 1] * point == 0) return false;
38     */
39     int l = 0, r = n - 1;
40     while (r - l > 1) {
41         int mid = (l + r) / 2;
42         int pos = mid;
43         if (seq[pos]*point >= 0) l = mid;
44         else r = mid;
45     }
46     int pos = l;
47     return pointInTriangle(seq[pos], seq[pos + 1], Point{0,0}, point);
48     /*
49     Mismo caso que no acepte cosas en el borde (Strictly inside)
50     return (seq[pos + 1] - seq[pos]) * (point - seq[pos]) > 0;
51     */
52 }
53
54 void doit(){
55     int n;
56     vector<Point> poly; //with input.
57     vector<Point> seq; //empty.
58     Point translation;
59     prepareConvexPolygon(n, poly, seq, translation);
60     // then call pointInConvexPolygon() for queries.
61 }
```

Point_Inside_Polygon

GEOMETRY · C++ · 18 LÍNEAS

CÓDIGO · Point_Inside_Polygon

```
1 // Checa si un punto dado esta DENTRO, FUERA o en FRONTERA con un poligono
2 // Recuerda cerrar el poligono
3 string checkPointInsidePolygon(vector <Point> P, Point point, int n){
4     P[0] = point;
5     ll count = 0;
6     if (n < 3) return "OUTSIDE";
7     for(int i = 1; i <= n; i++){
8         int j = (i == n ? 1 : i+1);
9         if(P[i].x <= P[0].x && P[0].x < P[j].x && P[0].cross(P[i], P[j]) < 0) count++;
10        else if(P[j].x <= P[0].x && P[0].x < P[i].x && P[0].cross(P[j], P[i]) < 0) count++;
11
12        if ((min(P[i].x,P[j].x) <= point.x && point.x <= max(P[i].x,P[j].x)) && (min(P[i].y,P[j].y) <=
13            point.y && point.y <= max(P[i].y,P[j].y)) && point.cross(P[i],P[j]) == 0){
14            return "BOUNDARY";
15        }
16    }
17    if (count%2 == 1) return "INSIDE";
18    return "OUTSIDE";
19 }
```

Polygon_Area

GEOMETRY · C++ · 6 LÍNEAS

```
# CÓDIGO · Polygon_Area
1 ld getPolygonArea(vector<Point> &poly){ //Calculo de area de poligono
2     ll ans = 0;
3     poly.push_back(poly.front());
4     for(int i = 1; i<(int)poly.size(); i++) ans += (poly[i-1]*poly[i]);
5     return abs(ans);
6 }
```

SECCIÓN 05

Graph



10 algoritmos

bellman_ford

GRAPH · C++ · 27 LÍNEAS

```
# CÓDIGO · bellman_ford

1  dis[start] = 0;
2  for (int i{}; i < n - 1; ++i) {
3      for (int node{}; node < n; ++node) {
4          if (dis[node] == INF) continue;
5          for (auto [u, w] : adj[node]) {
6              dis[u] = std::min(dis[u], dis[node] + w);
7          }
8      }
9  }
10 std::vector<int> ciclosos(n), q{};
11 for (int node{}; node < n; ++node) {
12     if (dis[node] == INF) continue;
13     for (auto [u, w] : adj[node]) {
14         if (dis[node] + w < dis[u]) {
15             ciclosos[u] = true;
16             q.push_back(u);
17         }
18     }
19 }
20 for (int j{}; j < std::ssize(q); ++j) {
21     auto node {q[j]};
22     for (auto [u, w] : adj[node]) {
23         if (ciclosos[u]) continue;
24         ciclosos[u] = true;
25         q.push_back(u);
26     }
27 }
```

binary_lifting

GRAPH · C++ · 30 LÍNEAS

```
# CÓDIGO · binary_lifting

1  #define ll long long
2  #define fore(i,a,b) for(int i = a; i<b; i++)
3  #define rofe(i,a,b) for(ll i=a-1;i>=b;--i)
4
5  const ll maxn = 2e5+5;
6  const ll loga = 20;
7  ll n, up[maxn][loga], dep[maxn];
8
9  void binlift(){
10     // Assuming we have all direct parents for each node (up[node][0])
11     fore(i,1,loga){
12         fore(j,1,n+1) up[j][i] = up[up[j][i-1]][i-1];
13     }
14 }
15
16 ll lca(ll x, ll y){ //calculate the depths of each node before.
17     if (dep[x] < dep[y]) swap(x,y);
18     ll dif = dep[x]-dep[y];
19     rofe(i,loga,0){
20         if (dif & (1ll<<i)) x = up[x][i];
21     }
22     if (x == y) return x;
23     rofe(i,loga,0){
24         if (up[x][i] != up[y][i]){
25             x = up[x][i];
26             y = up[y][i];
27         }
28     }
29     return up[x][0];
30 }
```

cycle_detection_directed_graph

GRAPH · C++ · 33 LÍNEAS

CÓDIGO · cycle_detection_directed_graph

```
1  enum Colors {none, explored, visited};
2  std::vector<int> vis(n);
3  auto hasCycle = [&](auto& self, int node) -> bool {
4      vis[node] = explored;
5      for (auto u : adj[node]) {
6          if (vis[u] == none) {
7              if (self(self, u)) return true;
8          } else if (vis[u] == explored) return true;
9      }
10     vis[node] = visited;
11     return false;
12 };
13
14 // kanh algorithm
15 std::vector<int> q{};
16 q.reserve(n);
17 for (int i{}; i < n; ++i) {
18     if (inDegree[i] == 0) {
19         q.push_back(i);
20     }
21 }
22 for (int j{}; j < std::ssize(q); ++j) {
23     auto node {q[j]};
24     for (auto u : adj[node]) {
25         --inDegree[u];
26         if (inDegree[u] == 0) {
27             q.emplace_back(u);
28         }
29     }
30 }
31 if (std::ssize(q) != n) {
32     std::cout << "There is a cycle.\n";
33 }
```

dijkstra

GRAPH · C++ · 24 LÍNEAS

```
# CÓDIGO · dijkstra

1  const long long INF = 5e18;
2  std::vector<int> dis(n, INF);
3
4  using T = std::pair<long long, int>;
5  std::priority_queue<T, std::vector<T>, std::greater<T>> pq{};
6
7  dis[start] = 0;
8  pq.emplace(dis[start], start);
9
10 std::vector<std::vector<std::pair<int,int>>> path(n);
11
12 while (!pq.empty()) {
13     auto [cost, node] {pq.top()}; pq.pop();
14     if (cost > dis[node]) continue;
15     for (auto [u, w] : adj[node]) {
16         if (dis[node] + w < dis[u]) {
17             dis[u] = dis[node] + w;
18             pq.emplace(dis[u], u);
19             path[u] = {std::pair{node, w}};
20         } else if (dis[node] + w == dis[u]) {
21             path[u].emplace_back(node, w);
22         }
23     }
24 }
```

floyd_warshall

GRAPH · C++ · 39 LÍNEAS

```
# CÓDIGO · floyd_warshall

1  const int nax {500};
2  std::array< std::array<int, nax>, nax> dis{}, p{};
3
4  const int INF {1'000'000'000};
5  for (auto& row : dis) row.fill(INF);
6  for (auto& row : p) row.fill(-1);
7
8  for (int b{}; b < n; ++b) {
9      for (int i{}; i < n; ++i) {
10         for (int j{}; j < n; ++j) {
11             if (dis[i][b] == INF || dis[b][j] == INF) continue;
12             if (dis[i][j] > dis[i][b] + dis[b][j]) {
13                 dis[i][j] = dis[i][b] + dis[b][j];
14                 p[i][j] = b;
15             }
16         }
17     }
18 }
19
20 int cycleSize{INF};
21 for (int i{}; i < n; ++i) {
22     cycleSize = std::min(cycleSize, dis[i][i]);
23 }
24
25 std::vector<int> ans{};
26 for (int I{}; I < n; ++I) {
27     if (dis[I][I] == cycleSize){
28         auto reconstruct = [&](auto& self, int i, int j) -> void {
29             if (p[i][j] == -1) {
30                 ans.push_back(i);
31                 return;
32             }
33             auto k {p[i][j]};
34             self(self, i, k);
35             self(self, k, j);
36         };
37         reconstruct(reconstruct, I, I);
38     }
39 }
```

lca

GRAPH · C++ · 41 LÍNEAS

```
# CÓDIGO · lca

1  const int LOG {20}, INF {1'000'000'000};
2
3  std::vector<int> depth(n, INF);
4  depth[0] = 0;
5
6  std::vector<std::vector<int>> up(n, std::vector<int> (LOG, -1));
7  auto dfs = [&](auto& self, int node) -> void {
8      for (auto u : adj[node]) {
9          if (depth[u] != INF) continue;
10         depth[u] = depth[node] + 1;
11         up[u][0] = node;
12         self(self, u);
13     }
14 };
15 dfs(dfs, 0);
16 for (int k{1}; k < LOG; ++k) {
17     for (int i{}; i < n; ++i) {
18         int m {up[i][k - 1]};
19         if (m != -1) {
20             up[i][k] = up[m][k - 1];
21         }
22     }
23 }
24 auto lca = [&](int a, int b) {
25     if (depth[a] < depth[b]) std::swap(a, b);
26     int k{depth[a] - depth[b]};
27
28     for (int i{}; i < LOG; ++i) {
29         if (k & (1 << i)) {
30             a = up[a][i];
31         }
32     }
33     if (a == b) return a;
34     for (int j{LOG - 1}; j >= 0; --j) {
35         if (up[a][j] != up[b][j]) {
36             a = up[a][j];
37             b = up[b][j];
38         }
39     }
40     return up[a][0];
41 }
```

mst

GRAPH · C++ · 33 LÍNEAS

```
# CÓDIGO · mst

1 // Kruskal
2 std::vector<std::tuple<int,int,int>> edges{};
3 std::ranges::sort(edges);
4
5 UF dsu{n};
6 std::vector<std::vector<int>> adj(n);
7 for (auto [weight, u, v] : edges) {
8     if (dsu.unite(u, v)) {
9         adj[u].push_back(v);
10        adj[v].push_back(u);
11    }
12    if (dsu.components() == 1) break;
13 }
14
15 // PRIM
16 using T = std::pair<int, int>;
17 std::priority_queue<T, std::vector<T>, std::greater<T>> pq{};
18 std::vector<int> vis(n);
19 auto process = [&](int node) {
20     vis[node] = true;
21     for (auto [u, w] : adj[node]) {
22         if (vis[u]) continue;
23         pq.emplace(w, u);
24     }
25 };
26 process(0);
27 long long cost{};
28 while (!pq.empty()) {
29     auto [w, node] {pq.top()}; pq.pop();
30     if (vis[node]) continue;
31     cost += w;
32     process(node);
33 }
```

tarjan

GRAPH · C++ · 73 LÍNEAS

```
# CÓDIGO · tarjan

1  const long long maxn = 2e5+10;
2  long long n,x,y,m,timer=0;
3  long long low[maxn],disc[maxn],isArt[maxn],inStack[maxn];
4  vector<long long> adj[maxn];
5  vector<vector<long long>> scc;
6  vector < pair<long long, long long> > brid;
7  stack <int> stk; // to find SCCs (directed graphs)
8  vector <pair<long long, long long>> edges; // to find BCCs (undirected graphs)
9
10 void extract_bcc_nodes(int stop_u, int stop_v, vector<long long> &bcc_nodes){
11     while((int)edges.size()){
12         auto e = edges.back();
13         edges.pop_back();
14         bcc_nodes.push_back(e.first);
15         bcc_nodes.push_back(e.second);
16         if ((e.first == stop_u && e.second == stop_v) || (e.first == stop_v && e.second == stop_u))
17             break;
18     }
19     sort(bcc_nodes.begin(), bcc_nodes.end());
20     bcc_nodes.erase(unique(bcc_nodes.begin(), bcc_nodes.end()), bcc_nodes.end());
21 }
22 void dfs(int node, int antnode = -1){
23     low[node] = disc[node] = ++timer;
24     stk.push(node);
25     inStack[node] = 1;
26     int children = 0;
27     for (auto signode : adj[node]){
28         if(disc[signode] == 0){
29             children++;
30             edges.push_back({node,signode});
31             dfs(signode, node);
32
33             // To find bridges:
34             if (low[signode] > disc[node]){
35                 brid.push_back({node,signode});
36             }
37
38             low[node] = min(low[node], low[signode]);
39
40             // To find articulation points (undirected graphs):
41             if (antnode == -1 && children > 1) isArt[node] = 1;
42             if (antnode != -1 && low[signode] >= disc[node]) isArt[node] = 1;
43
44             // To find BCCs (undirected graphs):
45             if (low[signode] >= disc[node]){
46                 vector<long long> bcc_nodes; // stores the nodes from a BCC (in no specific order)
47                 extract_bcc_nodes(node,signode,bcc_nodes);
48             }
49         }
50         // To find SCC's (directed graphs):
51         else if (inStack[signode] == 1){
52             low[node] = min(low[node], disc[signode]);
53         }
54         // To find articulation points, bridges, and/or BCCs (undirected graphs)
55         else if (antnode != signode){
56             if (disc[signode] < disc[node]){ // to find BCCs
57                 edges.push_back({node,signode});
58             }
59             low[node] = min(low[node], disc[signode]);
60         }
61     }
62     if (low[node] == disc[node]){ // To find SCC's (directed graphs)
63         vector<long long> scctem;
64         while (true){
65             long long topic = stk.top();
66             stk.pop();
```

```
# CÓDIGO · tarjan
67         scctem.push_back(topic);
68         inStack[topic] = 0;
69         if (node == topic) break;
70     }
71     scc.push_back(scctem);
72 }
73 }
```

tarjan_bridges_articulation

GRAPH · C++ · 32 LÍNEAS

```
# CÓDIGO · tarjan_bridges_articulation

1 // Undirected edges {vertex, id}
2 std::vector<std::vector<std::pair<int,int>>> adj(n);
3
4 std::vector<int> bridges(m), articulation(n);
5 int root{}, childrenRoot{};
6
7 std::vector<int> pre(n), low(n);
8 int cont{};
9 auto dfs = [&](auto& self, int node, int parentEdge) -> void {
10     pre[node] = low[node] = ++cont;
11     for (auto [u, id] : adj[node]) {
12         if (id == parentEdge) continue;
13         if (pre[u] == 0) {
14             // forward/tree edge
15             self(self, u, id);
16             if (node == root) ++childrenRoot;
17             if (low[u] > pre[node]) bridges[id] = true;
18             if (low[u] >= pre[node]) articulation[node] = true;
19             low[node] = std::min(low[u], low[node]);
20         } else {
21             // back edge
22             low[node] = std::min(low[node], pre[u]);
23         }
24     }
25 };
26
27 for (int i{}; i < n; ++i) {
28     if (pre[i]) continue;
29     root = i; childrenRoot = 0;
30     dfs(dfs, i, -1);
31     articulation[i] = (childrenRoot > 1);
32 }
```

tarjan_scc

GRAPH · C++ · 31 LÍNEAS

```
# CÓDIGO · tarjan_scc

1  const int INF {1'000'000'000};
2  std::vector<int> low(n), pre(n), id(n, -1);
3  std::stack<int> stk{};
4  int cont{}, numSCC{};
5
6  auto dfs = [&](auto& self, int node) -> void {
7      pre[node] = low[node] = ++cont;
8      stk.push(node);
9
10     for (auto u : adj[node]) {
11         if (pre[u] == 0) {
12             self(self, u);
13         }
14         low[node] = std::min(low[node], low[u]);
15     }
16     if (low[node] == pre[node]) {
17         while (true) {
18             auto x{stk.top()}; stk.pop();
19             id[x] = numSCC;
20             low[x] = INF;
21             if (x == node) break;
22         }
23         ++numSCC;
24     }
25 };
26 for (int i{}; i < n; ++i) {
27     if (pre[i] == 0) {
28         dfs(dfs, i);
29     }
30 }
31
```

SECCIÓN 06

Math



7 algoritmos

binary_exponentiation

MATH · C++ · 13 LÍNEAS

```
# CÓDIGO · binary_exponentiation

1  const int mod = 1e9 + 7;
2
3  auto power (long long a, long long exp) {
4      long long r{1};
5      while (exp) {
6          if (exp % 2) {
7              r = (a * r) % mod;
8          }
9          a = (a * a) % mod;
10         exp /= 2;
11     }
12     return r;
13 }
```

combinatorics

MATH · C++ · 34 LÍNEAS

CÓDIGO · combinatorics

```

1  const int nax {67};
2  auto pascal = []() {
3      std::array<std::array<long long, nax>, nax> p{};
4      p[0][0] = 1;
5      for (int i{1}; i < nax; ++i) {
6          p[i][0] = 1;
7          for (int j{1}; j < nax; ++j) {
8              p[i][j] += p[i - 1][j] + p[i - 1][j - 1];
9          }
10     }
11     return p;
12 }();
13
14 const int nax {100'000};
15
16 std::vector<long long> invFacto(nax, 1);
17 const auto facto = []() {
18     std::array<long long, nax> f;
19     f[0] = 1;
20     for (int i{1}; i < nax; ++i) {
21         f[i] = multi(f[i - 1], i);
22     }
23     invFacto[nax - 1] = divide(1, facto[nax - 1]);
24     for (int i{nax - 1}; i > 0; --i) {
25         invFacto[i - 1] = multi(invFacto[i], i);
26     }
27     return f;
28 }();
29
30 long long nCk(int n, int k) {
31     if (k < 0 || n < k) return 0;
32     return pascal[n][k];
33     return multi(facto[n], multi(invFacto[k], invFacto[n - k]));
34 }

```

fft

MATH · C++ · 141 LÍNEAS

#	CÓDIGO · fft
1	<code>typedef pair<long long, long long> ii;</code>
2	
3	<code>const long long MAXN=1<<20; //watch out with RTEs (increase MAXN):</code>
4	<code>typedef vector<long long> poly;</code>
5	<code>struct CD {</code>
6	<code> double r,i;</code>
7	<code> CD(double r=0, double i=0):r(r),i(i){}</code>
8	<code> double real()const{return r;}</code>
9	<code> void operator/=(const int c){r/=c, i/=c;}</code>
10	<code>};</code>
11	<code>CD operator*(const CD& a, const CD& b){</code>
12	<code> return CD(a.r*b.r-a.i*b.i, a.r*b.i+a.i*b.r);}</code>
13	<code>CD operator+(const CD& a, const CD& b){return CD(a.r+b.r, a.i+b.i);}</code>
14	<code>CD operator-(const CD& a, const CD& b){return CD(a.r-b.r, a.i-b.i);}</code>
15	<code>const double pi=acos(-1.0);</code>
16	
17	<code>CD cp1[MAXN+9], cp2[MAXN+9];</code>
18	<code>int R[MAXN+9];</code>
19	<code>void dft(CD* a, int n, bool inv){</code>
20	<code> for(int i = 0; i < n; i++) if(R[i]<i)swap(a[R[i]],a[i]);</code>
21	<code> for(int m=2; m<=n; m*=2){</code>
22	<code> double z=2*pi/m*(inv?-1:1);</code>
23	<code> CD wi=CD(cos(z), sin(z));</code>
24	<code> for(int j=0; j<n; j+=m){</code>
25	<code> CD w(1);</code>
26	<code> for(int k=j, k2=j+m/2; k2<j+m; k++, k2++){</code>
27	<code> CD u=a[k]; CD v=a[k2]*w; a[k]=u+v; a[k2]=u-v; w=w*wi;</code>
28	<code> }</code>
29	<code> }</code>
30	<code> }</code>
31	<code> if(inv) for(int i = 0; i < n; i++) a[i]/=n;</code>
32	<code>}</code>
33	<code>poly multiply(poly& p1, poly& p2){</code>
34	<code> int n=p1.size()+p2.size()+1;</code>
35	<code> int m=1, cnt=0;</code>
36	<code> while(m<=n)m*=m, cnt++;</code>
37	<code> for(int i = 0; i < m; i++){R[i]=0; for(int j = 0; j < cnt; j++) R[i]=(R[i]<<1) ((i>>j)&1);}</code>
38	<code> for(int i = 0; i < m; i++) cp1[i]=0, cp2[i]=0;</code>
39	<code> for(int i = 0; i < (int)p1.size(); i++) cp1[i]=p1[i];</code>
40	<code> for(int i = 0; i < (int)p2.size(); i++) cp2[i]=p2[i];</code>
41	<code> dft(cp1, m, false); dft(cp2, m, false);</code>
42	<code> for(int i = 0; i < m; i++) cp1[i]=cp1[i]*cp2[i];</code>
43	<code> dft(cp1, m, true);</code>
44	<code> poly res;</code>
45	<code> n-=2;</code>
46	<code> for(int i = 0; i < n; i++) res.push_back((long long)floor(cp1[i].real()+0.5));</code>
47	<code> return res;</code>
48	<code>}</code>
49	
50	<code>void getBigNumMulti(vector<long long> &c){ //Big numbers multiplication.</code>
51	<code> vector<char> r;</code>
52	<code> while(!c.empty()&&!c.back()) c.pop_back(); //quitar todos los 0 extras.</code>
53	<code> if(c.empty()){</code>
54	<code> cout<<0<<'\n';</code>
55	<code> return;</code>
56	<code> }</code>
57	<code> long long x=0;</code>
58	<code> //Normalizar los coeficientes para representarlos como digitos.</code>
59	<code> for(int i = 0; i < (int)c.size(); i++){</code>
60	<code> x+=c[i];</code>
61	<code> r.push_back((char)(x%10) + '0');</code>
62	<code> x/=10;</code>
63	<code> }</code>
64	<code> while(x){ //carry que sobra.</code>
65	<code> r.push_back((char)(x%10) + '0');</code>
66	<code> x/=10;</code>

```

# CÓDIGO · fft
67     }
68     reverse(r.begin(), r.end());
69     for(int i = 0; i < (int)r.size(); i++) cout<<r[i];
70     cout<<'\n';
71 }
72
73 void stringMatchShift(){ //All possible scalar products with strings.
74     string s;
75     cin>>s;
76     int n = s.size();
77     vector<long long> a1(n,0),a2(2*n,0),b1(n,0),b2(2*n,0),c1(n,0),c2(2*n,0);
78     vector<long long> ra,rb,rc;
79
80     //Create binary polynomial for each letter.
81     for(int i = 0; i < n; i++){
82         if (s[i] == 'a') a1[i] = 1;
83         else if (s[i] == 'b') b1[i] = 1;
84         else c1[i] = 1;
85     }
86     //Make the dup for each letter to multiply:
87     for(int i = 0; i < n; i++){
88         a2[i] = a2[i+n] = a1[i];
89         b2[i] = b2[i+n] = b1[i];
90         c2[i] = c2[i+n] = c1[i];
91     }
92     //Append the rest of the zeros (Step 1):
93     for(int i = 0; i < n; i++){
94         a1.push_back(0), b1.push_back(0), c1.push_back(0);
95     }
96     //Reverse the arrays (Step 2):
97     reverse(a1.begin(), a1.end());
98     reverse(b1.begin(), b1.end());
99     reverse(c1.begin(), c1.end());
100    //Multiply the polynomials:
101    ra = multiply(a1,a2);
102    rb = multiply(b1,b2);
103    rc = multiply(c1,c2);
104    int shif = 1;
105    //Left shift match (Step 1, then Step 2):
106    for(int i = (2*n)-2; i>=n; i--){
107        cout<<"L-shift: "<<shif<<" "<<ra[i] + rb[i] + rc[i]<<'\n';
108        shif++;
109    }
110    //Right shift match (Step 2, then Step 1):
111    shif = 1;
112    for(int i = n-2; i>=0; i--){
113        cout<<"R-shift: "<<shif<<" "<<ra[i] + rb[i] + rc[i]<<'\n';
114        shif++;
115    }
116 }
117
118 //String matching with wildcards ('?')
119 vector<long long> string_matching(string &s, string &t) {
120     int n = s.size(), m = t.size();
121     vector<long long> s1(n), s2(n), s3(n);
122     //assign any non zero number for non '?'s
123     for(int i = 0; i < n; i++) s1[i] = s[i] == '?' ? 0 : s[i] - 'a' + 1;
124     for(int i = 0; i < n; i++) s2[i] = s1[i] * s1[i];
125     for(int i = 0; i < n; i++) s3[i] = s1[i] * s2[i];
126     vector<long long> t1(m), t2(m), t3(m);
127     for(int i = 0; i < m; i++) t1[i] = t[i] == '?' ? 0 : t[i] - 'a' + 1;
128     for(int i = 0; i < m; i++) t2[i] = t1[i] * t1[i];
129     for(int i = 0; i < m; i++) t3[i] = t1[i] * t2[i];
130     reverse(t1.begin(), t1.end());
131     reverse(t2.begin(), t2.end());
132     reverse(t3.begin(), t3.end());
133     vector<long long> s1t3 = multiply(s1, t3);
134     vector<long long> s2t2 = multiply(s2, t2);
135     vector<long long> s3t1 = multiply(s3, t1);
136     vector<long long> res(n);
137     for(int i = 0; i < n; i++) res[i] = s1t3[i] - s2t2[i] * 2 + s3t1[i];
138     vector<long long> oc;

```

```
# CÓDIGO · fft
139     for(int i = m - 1; i < n; i++) if(res[i] == 0) oc.push_back(i - m + 1);
140     return oc;
141 }
```

matrix_expo

MATH · C++ · 30 LÍNEAS

```
# CÓDIGO · matrix_expo

1  const long long mod = 1e9+7;
2  long long tc,n,m,k;
3
4  vector<vector<long long>> mul(vector<vector<long long>> a, vector<vector<long long>> b) {
5      vector<vector<long long>> c((int)a.size(), vector<long long>((int)b[0].size()));
6      for (int i = 0; i < (int)a.size(); i++)
7          for (int j = 0; j < (int)b.size(); j++)
8              for (int k = 0; k < (int)a.size(); k++)
9                  (c[i][j] += a[i][k]*b[k][j]%mod)%=mod; //for amount of paths.
10                 //c[i][j] = min(c[i][j], a[i][k] + b[k][j]); //for shortest path.
11     return c;
12 }
13
14 vector<vector<long long>> exp( vector<vector<long long>> x, int y) { // matrix and desired power.
15     vector<vector<long long>> r((int)x.size(), vector<long long>((int)x.size(),0ll)); //0ll: amount of
16     paths. INF: shortest path
17     for (int i = 0; i < (int)x.size(); i++) r[i][i] = 1; //1ll: amount of paths. 0ll: shortest path.
18     while (y>0){
19         if (y&1) {
20             r = mul(r,x);
21         }
22         y=y>>1;
23         x = mul(x,x);
24     }
25     return r;
26 }
27 void doit(){
28     // build adjacency (or costs) matrix of size(n*n).
29     // after exponentiating mat[i][j] denotes the path from i to j.
30 }
```

modular_arithmetic

MATH · C++ · 20 LÍNEAS

```
# CÓDIGO · modular_arithmetic

1  const int mod = 1e9 + 7;
2  long long power(long long a, long long exp);
3
4  auto toMod(long long x) {
5      return ((x % mod) + mod) % mod;
6  }
7  auto add(long long a, long long b) {
8      auto ans {a + b};
9      return (ans >= mod) ? ans - mod : ans;
10 }
11 auto sub(long long a, long long b) {
12     auto ans {a - b};
13     return (ans < 0) ? ans + mod : ans;
14 }
15 auto multi(long long a, long long b) {
16     return (a * b) % mod;
17 }
18 auto divide(long long a, long long b) {
19     return multi(a, power(b, mod - 2));
20 }
```

precision

MATH · C++ · 17 LÍNEAS

```
# CÓDIGO · precision

1 constexpr double eps {1e-9};
2
3 bool equal(double a, double b) {
4     if (std::abs(a - b) <= eps) return true;
5     return std::abs(a - b) <= std::max(std::abs(a), std::abs(b)) * eps;
6 }
7 bool less(double a, double b) {
8     return a < (b - eps);
9 }
10 bool lessEqual(double a, double b) {
11     return a < (b + eps);
12 }
13 long long divCeil(long long x, long long y) {
14     bool neg {(x < 0) != (y < 0)};
15     if (neg) return x / y;
16     return (x / y) + (x % y != 0);
17 }
```

prime_theory

MATH · C++ · 64 LÍNEAS

```
# CÓDIGO · prime_theory

1  std::vector<long long> primes{};
2  const int nax {100'000};
3
4  const auto gc = []() {
5      std::bitset<nax + 1> c{};
6      c[0] = c[1] = true;
7      primes.reserve((nax / std::log(nax)) * 1.2);
8
9      for (int m{4}; m <= nax; m += 2) c[m] = true;
10     primes.push_back(2);
11     for (long long p{3}; p <= nax; p += 2) {
12         if (c[p]) continue;
13         primes.push_back(p);
14         for (long long m{p * p}; m <= nax; m += p + p) {
15             c[m] = true;
16         }
17     }
18     return c;
19 }();
20 bool isPrime(long long m) {
21     if (m <= nax) return !gc[m];
22     for (auto p : primes) {
23         if (p * p > m) break;
24         if (m % p == 0) return false;
25     }
26     return true;
27 }
28 auto primeFactors(long long m) {
29     std::vector<std::pair<long long,int>> f{};
30     for (auto p : primes) {
31         if (p * p > m) break;
32         int cont{};
33         while (m % p == 0) {
34             ++cont;
35             m /= p;
36         }
37         if (cont) f.emplace_back(p, cont);
38     }
39     if (m != 1) f.emplace_back(m, 1);
40     return f;
41 }
42 auto eulerPhiSieve = []() {
43     std::array<long long, nax + 1> a{};
44     std::iota(a.begin(), a.end(), 0);
45     for (int i{2}; i <= nax; ++i) {
46         if (a[i] == i) {
47             for (int j{i}; j <= nax; j += i) {
48                 a[j] = (a[j] / i) * (i - 1);
49             }
50         }
51     }
52     return a;
53 }();
54 auto eulerPhi(long long n) {
55     if (n <= nax) return eulerPhiSieve[n];
56     long long ans{n};
57     for (auto p : primes) {
58         if (p * p > n) break;
59         if (n % p == 0) ans -= ans / p;
60         while (n % p == 0) n /= p;
61     }
62     if (n != 1) ans -= ans / n;
63     return ans;
64 }
```

SECCIÓN 07

Misc



2 algoritmos

jose

MISC · C++ · 22 LÍNEAS

```
# CÓDIGO · jose

1 int J(int n, int k) {
2     if (n == 1) return 0; // 1 (1-index)
3     return (J(n - 1, k) + k) % n;
4 }
5
6 int josephus(int n, int k) {
7     int res = 0;
8     for (int i = 1; i <= n; ++i){
9         res = (res + k) % i;
10    }
11    return res; // + 1 (1-index)
12 }
13
14 std::vector<int> ans{};
15 int pos{1};
16 // 1-index
17 for (int i{}; i < n; ++i) {
18     int r {n - i};
19     pos = ((pos + k - 1) % r) + 1;
20     ans.push_back(jose.find(pos));
21     jose.kill(ans.back());
22 }
```

random

MISC · C/C++ Header · 20 LÍNEAS

```
# CÓDIGO · random
1  #ifndef RANDOM_H
2  #define RANDOM_H
3
4  #include <chrono>
5  #include <random>
6
7  namespace Random {
8      inline std::mt19937_64 mt {
9          static_cast<std::seed_seq::result_type> (
10             std::chrono::steady_clock::now().time_since_epoch().count()
11         )
12     };
13
14     template<typename T>
15     T get(T min, T max) {
16         return std::uniform_int_distribution<T>{min, max}(mt);
17     }
18 }
19
20 #endif
```

SECCIÓN 08

String



6 algoritmos

aho_corasick

STRING · C++ · 48 LÍNEAS

```
# CÓDIGO · aho_corasick

1  const int MAXN = 1e6+10;
2  map<char, int> to[MAXN]; // if TLE change to array
3  string a[MAXN];
4  int lnk[MAXN], sz=1;
5  long long que[MAXN], endlink[MAXN];
6  vector<long long> leaf[MAXN], ans[MAXN];
7  void add_str(string s, int id) {
8      int v = 0;
9      for(char c: s) {
10         if(!to[v].count(c)) to[v][c] = sz++;
11         v = to[v][c];
12     }
13     leaf[v].push_back(id); //Node in automata where a word ends
14 }
15 void push_links() {
16     queue<int> q({0});
17     while(!q.empty()) {
18         int v = q.front(); q.pop();
19         for(auto [c,u]: to[v]){
20             int j = lnk[v];
21             while(j && !to[j].count(c)) j=lnk[j];
22             if(to[j].count(c) && to[j][c]!=u) lnk[u] = to[j][c];
23             endlink[u] = leaf[lnk[u]].size()>0?lnk[u]:endlink[lnk[u]];
24             q.push(u);
25         }
26     }
27 }
28 void walk(string s){ //KMP with multiple target patterns
29     int v=0;
30     for(int i = 0; i < (int)s.size(); i++){
31         char c=s[i];
32         while(v && !to[v].count(c)) v=lnk[v];
33         if(to[v].count(c)) v=to[v][c];
34         for(int u=v; u=endlink[u];) for(int x: leaf[u]){
35             ans[x].push_back(i); //pushing the index of the main string where a pattern ends
36         }
37     }
38 }
39 void doit(){
40     string s;
41     cin>>s; //main string.
42     for(int i = 0; i < n; i++){
43         cin>>a[i];
44         add_str(a[i], i); //add target strings.
45     }
46     push_links();
47     walk(s);
48 }
```

hashing

STRING · C++ · 66 LÍNEAS

CÓDIGO · hashing

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // To hash a multiset: Sum of elements of: (B^num) * freq[num]
5  const long long MOD = 1000000009LL; // prime modulo (alternative: 10000000000000061 or 1000000000000061)
6  const long long BASE = 131LL; // any number > alphabet size and coprime to MOD.
7
8  string s, t;
9  vector<long long> pws, h;
10 long long hashT = 0, ans = 0;
11
12 long long convertChar(char c) {
13     return static_cast<long long>(c - 'a' + 1);
14 }
15
16 bool sameHash(int l1, int len1, int l2, int len2) {
17     int r1 = l1 + len1;
18     int r2 = l2 + len2;
19     long long h1 = (h[r1] - h[l1] * pws[len1] % MOD + MOD) % MOD;
20     long long h2 = (h[r2] - h[l2] * pws[len2] % MOD + MOD) % MOD;
21     return h1 == h2;
22 }
23
24 void precalc() {
25     int lenS = static_cast<int>(s.size());
26     int lenT = static_cast<int>(t.size());
27
28     pws.assign(lenS + 1, 1);
29     h.assign(lenS + 1, 0);
30
31     for (int i = 1; i <= lenS; ++i) {
32         pws[i] = (pws[i - 1] * BASE) % MOD;
33     }
34
35     for (int i = 0; i < lenS; ++i) {
36         h[i + 1] = (h[i] * BASE + convertChar(s[i])) % MOD;
37     }
38
39     for (int i = 0; i < lenT; ++i) {
40         hashT = (hashT * BASE + convertChar(t[i])) % MOD;
41     }
42 }
43
44 void doit() {
45     int lenS = static_cast<int>(s.size());
46     int lenT = static_cast<int>(t.size());
47
48     precalc();
49
50     for (int i = lenT; i <= lenS; ++i) {
51         long long curHash = (h[i] - h[i - lenT] * pws[lenT] % MOD + MOD) % MOD;
52         if (curHash == hashT) {
53             ++ans;
54         }
55     }
56 }
57
58 int main() {
59     ios::sync_with_stdio(false);
60     cin.tie(nullptr);
61
62     cin >> s >> t;
63     doit();
64     cout << ans << '\n';
65     return 0;
66 }

```

kmp

STRING · C++ · 40 LÍNEAS

```
# CÓDIGO · kmp

1  vector<long long> prefix_function(string s){
2      long long n= (int)s.size();
3      vector<long long> pi(n);
4
5      for(int i = 1; i<n; i++){
6          long long j=pi[i-1];
7
8          while(j>0 && s[i]!=s[j])
9              j=pi[j-1];
10
11         if(s[i]==s[j])
12             j++;
13
14         pi[i]=j;
15     }
16
17     return pi;
18 }
19
20 vector<long long> kmp(string s,string pat){
21     vector<long long> pi=prefix_function(pat);
22     vector<long long> ans;
23
24     long long j=0;
25
26     for(int i = 0; i< (int)s.size(); i++){
27         while(j>0 && s[i]!=pat[j])
28             j=pi[j-1];
29
30         if(s[i]==pat[j])
31             j++;
32
33         if(j==sz(pat)){
34             ans.pb(i-sz(pat)+1);
35             j=pi[j-1];
36         }
37     }
38
39     return ans;
40 }
```

suffix_array

STRING · C++ · 308 LÍNEAS

```
# CÓDIGO · suffix_array

1 // Push back a sentinel {static_cast<char>(1)} to the string before SuffixArray
2 class SuffixArray {
3 public:
4     SuffixArray (std::string_view a)
5         : s{a}, n{static_cast<int>(std::ssize(a))},
6         suffix(n), rank(n), lcp(n) {
7         build();
8         buildlcp();
9     }
10    int size() { return n; }
11    const auto& get_lcp() { return lcp; }
12    const auto& get_suffix() { return suffix; }
13 private:
14    std::string_view s{};
15    int n{};
16    std::vector<int> suffix{}, rank{}, lcp{};
17
18    void csort(int k) {
19        int m{std::max(n, 300)};
20        std::vector<int> f(m);
21        for (int i{}; i < n; ++i) {
22            ++f[(i + k < n) ? rank[i + k] : 0];
23        }
24        for (int i{1}; i < m; ++i) f[i] += f[i - 1];
25        std::vector<int> b{suffix.rbegin(), suffix.rend()};
26        for (auto u : b) {
27            int r{(u + k < n) ? rank[u + k] : 0};
28            suffix[--f[r]] = u;
29        }
30    }
31    void build() {
32        std::iota(suffix.begin(), suffix.end(), 0);
33        for (int i{}; i < n; ++i) rank[i] = s[i];
34
35        for (int k{1}; k < n; k *= 2) {
36            csort(k);
37            csort(0);
38            std::vector<int> tr(n);
39            int r{};
40            tr[suffix[0]] = r;
41            for (int i{1}; i < n; ++i) {
42                int cur {suffix[i]};
43                int prev {suffix[i - 1]};
44                tr[cur] = (
45                    (rank[cur] == rank[prev])
46                    && (rank[cur + k] == rank[prev + k])
47                    ) ? r : ++r;
48            }
49            std::swap(rank, tr);
50            if (rank[suffix[n - 1]] == n - 1) break;
51        }
52    }
53    void buildlcp() {
54        std::vector<int> phi(n), plcp(n);
55        phi[suffix[0]] = -1;
56        for (int i{1}; i < n; ++i) phi[suffix[i]] = suffix[i - 1];
57        for (int i{}, len{}; i < n; ++i) {
58            if (phi[i] == -1) {
59                plcp[i] = 0;
60                continue;
61            }
62            while ((i + len < n) && (phi[i] + len < n)
63                && (s[i + len] == s[phi[i] + len])) ++len;
64            plcp[i] = len;
65            len = std::max(len - 1, 0);
66        }
67    }
68 }
```

```

# CÓDIGO · suffix_array

67     for (int i{}; i < n; ++i) {
68         lcp[i] = plcp[suffix[i]];
69     }
70 }
71 };
72
73 // Longest-Repeated-Substring O(N)
74 // Returns {length, start_index} of one occurrence (lexicographically smallest).
75 std::pair<int,int> lrs (SuffixArray& sa) {
76     const auto& lcp {sa.get_lcp()};
77     const auto& suffix {sa.get_suffix()};
78
79     std::pair<int,int> ans{-1, -1};
80     for (int i{}; i < sa.size(); ++i) {
81         if (lcp[i] > ans.first) {
82             ans = {lcp[i], suffix[i]};
83         }
84     }
85     return ans;
86 }
87
88 // Longest-Repeated-Substring-Occurrence-Count O(N)
89 // Finds the longest substring that repeats, and how many times it occurs.
90 // Returns {length, occurrence_count} of that substring.
91 // If no substring repeats, returns {0, 1} (every substring occurs once).
92 std::pair<int,int> lrso(SuffixArray& sa) {
93     const auto& lcp{sa.get_lcp()};
94     int m{std::ssize(lcp)};
95
96     int len{};
97     for (int x : lcp) len = std::max(len, x);
98     if (len == 0) return {0, 1};
99
100    int ans{1}, run{};
101    for (int i{}; i < m; ++i) {
102        if (lcp[i] >= len) {
103            ++run;
104            ans = std::max(ans, run + 1);
105        } else {
106            run = 0;
107        }
108    }
109    return {len, ans};
110 }
111
112 // Count-Distinct-Substrings O(N)
113 long long cds (SuffixArray& sa) {
114     long long n{sa.size() - 1}, ans {(n * (n + 1)) / 2};
115     for (auto u : sa.get_lcp()) ans -= u;
116     return ans;
117 }
118
119 // Count-Distinct-Substrings-Repetead-At-Least-K O(N)
120 long long cdrk (SuffixArray& sa, int k) {
121     const auto& lcp{sa.get_lcp()};
122     int n {std::ssize(lcp)};
123     if (k < 1 || n < k) return 0;
124     if (k == 1) {
125         return cds(sa);
126     }
127
128     std::deque<int> dq{};
129     long long ans{};
130     int x{};
131
132     for (int i{1}; i < n; ++i) {
133         while (!dq.empty() && lcp[dq.back()] >= lcp[i]) dq.pop_back();
134         dq.push_back(i);
135         if (dq.front() <= i - (k - 1)) dq.pop_front();
136         int l{i - (k - 2)};
137         if (l < 1) continue;
138

```

```

# CÓDIGO · suffix_array

139     ans += std::max(0, lcp[dq.front()] - x);
140     x = lcp[dq.front()];
141 }
142 return ans;
143 }
144
145 std::string kth_distinct_substring(SuffixArray& sa, const std::string& s, long long k) {
146     const auto& suf = sa.get_suffix();
147     const auto& lcp = sa.get_lcp();
148     const int n = std::ssize(s) - 1;
149
150     for (int i{}; i < std::ssize(suf); ++i) {
151         long long prev = (i == 0) ? 0 : lcp[i];
152         long long m = (n - suf[i]) - prev;
153         if (m <= 0) continue;
154         if (k <= m) return s.substr(suf[i], prev + k);
155         k -= m;
156     }
157     return "";
158 }
159
160 // Longest-Repeat-Substring-At-Least-K O(N)
161 // Returns {length, start_index} of one valid occurrence – specifically
162 // the occurrence with the largest/smallest starting index among all substrings
163 std::pair<int,int> lrsk(SuffixArray& a, int k) {
164     int n{a.size()};
165     int m{k - 1};
166     if (m > n - 1) return {0, 0};
167     if (k <= 1) return {n - 1, 0};
168
169     const auto& lcp {a.get_lcp()};
170     const auto& suffix {a.get_suffix()};
171     std::deque<int> dq, dqpos{};
172
173     int best{}, idx{-1};
174     for (int i{}; i < n; ++i) {
175         while (!dq.empty() && lcp[dq.back()] >= lcp[i]) dq.pop_back();
176         dq.push_back(i);
177         if (dq.front() <= i - m) dq.pop_front();
178
179         while (!dqpos.empty() && suffix[dqpos.back()] <= suffix[i]) dqpos.pop_back();
180         dqpos.push_back(i);
181         if (dqpos.front() < i - m) dqpos.pop_front();
182
183         if (i >= m) {
184             int maxpos {suffix[dqpos.front()]};
185             if (best < lcp[dq.front()]) {
186                 best = lcp[dq.front()];
187                 idx = maxpos;
188             } else if (best == lcp[dq.front()] && best > 0) {
189                 idx = std::max(idx, maxpos);
190             }
191         }
192     }
193     if (best == 0) return {};
194     return {best, idx};
195 }
196
197 // Longest-Repeat-Substring-By-Lenght O(N)
198 // Returns a vector of size n+1; index L (1..n) holds max occurrences of
199 // length-L substrings.
200 std::vector<int> lrbl(SuffixArray& sa) {
201     int n {sa.size()};
202     const auto& lcp {sa.get_lcp()};
203     std::vector<int> left(n), right(n), stk{-1};
204     stk.reserve(n + 1);
205
206     for (int i{}; i < n; ++i) {
207         while (stk.back() != -1 && lcp[stk.back()] >= lcp[i]) stk.pop_back();
208         left[i] = stk.back();
209         stk.push_back(i);
210     }

```

```

# CÓDIGO · suffix_array

211
212     stk.clear();
213     stk.push_back(n);
214     for (int i{n - 1}; i >= 0; --i) {
215         while (stk.back() != n && lcp[stk.back()] >= lcp[i]) stk.pop_back();
216         right[i] = stk.back();
217         stk.push_back(i);
218     }
219
220     std::vector<int> ans(n + 1, 1);
221     for (int i{}; i < n; ++i) {
222         int width = right[i] - left[i] - 1;
223         ans[lcp[i]] = std::max(ans[lcp[i]], width + 1);
224     }
225
226     for (int i{n - 1}; i >= 1; --i) {
227         ans[i] = std::max(ans[i], ans[i + 1]);
228     }
229     return ans;
230 }
231
232 // LCP(i, j): longest common prefix of suffixes starting at i, j.
233 // i == j ---> n - i (whole suffix, no RMQ).
234 // i != j ---> let lo,hi = minmax(rank[i], rank[j]); RMQ(lcp[lo+1..hi]).
235
236 // Helper function to concatend strings for lcks
237 // preprocess the suffix_belongs
238 // Return string, ready to SuffixArray
239 int offset {105};
240 std::vector<int> suffix_belongs{};
241 std::vector<int> concatenated_strings (std::vector<std::string>& s) {
242     std::vector<int> a{};
243     suffix_belongs.clear();
244     int maxn{};
245     for (auto& u : s) maxn += std::ssize(u) + 1;
246     a.reserve(maxn);
247     suffix_belongs.reserve(maxn);
248
249     for (int i{}; i < std::ssize(s); ++i) {
250         for (auto c : s[i]) {
251             a.push_back(c + offset);
252             suffix_belongs.push_back(i);
253         }
254         a.push_back(static_cast<char>(1 + i));
255         suffix_belongs.push_back(-1);
256     }
257     return a;
258 }
259
260 // Longest-Common-Substring-At-Least-K-Different Strings
261 // Finding the longest common substring that appears in at least K
262 // different strings out of a given collection of strings.
263 // Return the max {length, vector<>{index}}
264 // Most of the time should work with integers and offset them to a wider range
265 // 1->(k + 1) sentinel char, A = (k + 1) -> Z = (k + 1 + 'A')
266 std::pair<int, std::vector<int>> lcsk (SuffixArray& sa, int k) {
267     int n {sa.size()};
268     const auto& lcp {sa.get_lcp()};
269     const auto& suffix {sa.get_suffix()};
270
271     std::unordered_map<int, int> f{};
272     int distinct{};
273     std::deque<int> dq{};
274
275     auto addSuffix = [&](int i) {
276         if (i < 0) return;
277         if (f[i]++ == 0) ++distinct;
278     };
279     auto rmSuffix = [&](int i) {
280         if (i < 0) return;
281         if (--f[i] == 0) --distinct;
282     };

```

```
# CÓDIGO · suffix_array

283     };
284
285     std::pair<int, std::vector<int>> ans{};
286
287     int l{};
288     for (int i{}; i < n; ++i) {
289         addSuffix(suffix_belongs[suffix[i]]);
290         while ((!dq.empty()) && lcp[dq.back()] >= lcp[i]) dq.pop_back();
291         if (i > l) dq.push_back(i);
292
293         while (distinct >= k) {
294             if (!dq.empty()) {
295                 int len {lcp[dq.front()]};
296                 if (len > ans.first) {
297                     ans = {len, std::vector{suffix[l]}};
298                 } else if (len == ans.first && len > 0) {
299                     ans.second.push_back(suffix[l]);
300                 }
301             }
302             rmSuffix(suffix_belongs[suffix[l]]);
303             if ((!dq.empty()) && dq.front() == l + 1) dq.pop_front();
304             ++l;
305         }
306     }
307     return ans;
308 }
```

Trie

STRING · C++ · 64 LÍNEAS

```
# CÓDIGO · Trie

1 struct Trie {
2     struct Node {
3         int nxt[26];
4         bool end;
5
6         Node() {
7             fill(nxt, nxt + 26, -1);
8             end = false;
9         }
10    };
11
12    vector<Node> tr;
13
14    Trie() {
15        tr.emplace_back();
16    }
17
18    void insert(const string& s) {
19        int u = 0;
20
21        for (char c : s) {
22            int x = c - 'a';
23
24            if (tr[u].nxt[x] == -1) {
25                tr[u].nxt[x] = (int)tr.size();
26                tr.emplace_back();
27            }
28
29            u = tr[u].nxt[x];
30        }
31
32        tr[u].end = true;
33    }
34
35    bool find(const string& s) {
36        int u = 0;
37
38        for (char c : s) {
39            int x = c - 'a';
40
41            if (tr[u].nxt[x] == -1)
42                return false;
43
44            u = tr[u].nxt[x];
45        }
46
47        return tr[u].end;
48    }
49
50    bool prefix(const string& s) {
51        int u = 0;
52
53        for (char c : s) {
54            int x = c - 'a';
55
56            if (tr[u].nxt[x] == -1)
57                return false;
58
59            u = tr[u].nxt[x];
60        }
61
62        return true;
63    }
64};
```

z_funtion

STRING · C++ · 21 LÍNEAS

```
# CÓDIGO · z_funtion

1  vector<long long> z_function(string s){
2      ll n=(int)s.size();
3      vector<long long> z(n);
4      long long l=0,r=0;
5
6      for(int i = 1; i<n ; i++){
7          if(i<=r)
8              z[i]=min(r-i+1,z[i-l]);
9
10         while(i+z[i]<n && s[z[i]]==s[i+z[i]])
11             z[i]++;
12
13         if(i+z[i]-1>r){
14             l=i;
15             r=i+z[i]-1;
16         }
17     }
18
19     return z;
20 }
21
```